

# Runbook — Promote Captain Adel to its own repo (``iflygaca/Captain-Adel``)

SECTION 06-operations-it

TYPE runbook

STATUS **active**

OWNER Founder

UPDATED 2026-06-16

**STATUS — DONE (2026-06-13).** Captain Adel was split out to `iflygaca/Captain-Adel` (as a snapshot, not a `git subtree split`) and the `captadel/` subtree was removed from this repo. The procedure below is kept for reference. Note the live repo is named `Captain-Adel`, not `captadel`, and history was not preserved.

The Captain Adel service lives in this repo as an **extraction-ready subtree** at `captadel/`. Everything it needs — server, brain, corpus, evals, Dockerfile, its own CI workflow — sits under that one prefix and references nothing outside it. This runbook splits that subtree into a standalone `iflygaca/Captain-Adel` repository (for `captadel.com`), preserving the option to keep both in sync.

Fly GACA does **not** depend on the subtree staying here: the `chat` Cloud Function already calls the service over the network (`ADEL_API_URL`), so once `captadel` is deployed from its own repo, the `captadel/` copy here can stay as a mirror or be retired later — that's a separate decision, not part of this cut.

Run this from a normal authenticated clone on your machine — it can't be done from the Claude Code web environment, whose Git/GitHub access is scoped to `flygaca/flygaca`.

## What you need first

1. **Org rights** to create a repository under the `FlyGACA` organization.
2. The **GitHub CLI** (`gh auth login`) — or just use the web UI for step 2.
3. A clean **clone of** `flygaca/flygaca`. Recommended: do this **after PR #18 is merged to** `main`, so the standalone repo's history is seeded from `main` (the canonical source) rather than a feature branch.

## One-time extraction

**Create the repo EMPTY.** Do **not** add a README, `.gitignore`, or license at creation. The subtree already ships those, and any initial commit GitHub makes would block the `subtree split` push below. With `gh repo create` you're fine (it makes an empty repo); in the **web UI**, leave "Add a README", "Add .gitignore", and "Choose a license" all **unchecked**.

```
# 0. From your flygaca/flygaca clone, get on the seed commit.
git checkout main && git pull          # (or the branch you want to seed from)

# 1. Create the EMPTY repo (or do this in the GitHub web UI). Keep it private -
#     it carries the GACAR corpus + system prompt and serves PDPL-sensitive traffic.
gh repo create iflygaca/Captain-Adel --private \
  --description "Captain Adel - independent AI flight instructor for Saudi civil
aviation (captadel.com). Standalone RAG service and the brain Fly GACA plugs into." \
  --homepage "https://captadel.com"

# 2. Split the captadel/ subtree into a branch whose root IS captadel/.
#     This produces a clean history containing only the commits that touched
#     captadel/, with the prefix stripped (src/, evals/, package.json ... at root).
git subtree split --prefix=captadel -b captadel-export

# 3. Push that history as the new repo's main.
git push https://github.com/iflygaca/Captain-Adel.git captadel-export:main

# 4. Tidy up the local export branch.
git branch -D captadel-export
```

## In the new repo — un-nest CI

The service ships its own workflow at `captadel/.github/workflows/ci.yml`. While it lived inside this monorepo it was scoped to the subtree; once `captadel` is the repo root, drop that scoping (the workflow's own header comment says so):

```
git clone https://github.com/iflygaca/Captain-Adel.git
cd Captain-Adel
```

In `.github/workflows/ci.yml`:

- remove the `on.push.paths:` and `on.pull_request.paths:` filters (the `- 'captadel/**'` lines), so it triggers on every push/PR; and
- remove the `defaults: { run: { working-directory: captadel } }` block, since the repo root is now `captadel/`.

Commit and push:

```
git add .github/workflows/ci.yml
git commit -m "ci: un-nest workflow now that captadel is the repo root"
git push
```

---

## GitHub repo settings (after the first push)

---

- **About → Description / Website:** already set in step 1 (the captadel.com line). If you created the repo via the web UI instead, paste the description there and set the website to `https://captadel.com`.
  - **About → Topics:** `aviation`, `gacar`, `saudi-arabia`, `rag`, `llm`, `gemini`, `allam`, `express`, `cloud-run`.
  - **Settings → Secrets and variables → Actions →** `GEMINI_API_KEY`: add it so the live `eval` job runs. The `--dry` structure check stays green without it, so forks/key-less branches don't fail.
  - **Settings → Branches → branch protection on** `main`: require a PR and the `captain-adel ci` check before merge.
  - Optional: turn off **Wiki** and **Projects** under Settings if unused.
- 

## Configure & verify

---

```
# Deps + the offline checks (no API key needed):
npm install
npm run smoke          # server module loads
npm run eval:dry       # cases.json validates
node evals/checks/citation-faithfulness.js --selftest
docker build -t captadel . # optional: container builds

# Live, when you have a key / endpoint:
GEMINI_API_KEY=... npm run eval
ALLAM_BASE_URL=... npm run eval:allam # needs a GPU vLLM/TGI endpoint
```

Set the service's runtime config from `.env.example` ( `GEMINI_API_KEY`, `MODEL_PROVIDER`, `ADEL_API_KEY`, `ALLOWED_ORIGINS`, optional `ALLAM_*` ). Deploy to a **KSA region** (Cloud Run `me-central1`) — real user questions are personal data under the PDPL; never the EU VPS. See `README.md` and `deploy/allam-vllm.md`.

When Fly GACA points at the deployed service, set the gateway's secrets in `flygaca/flygaca`: `ADEL_API_URL` (the service URL) and `ADEL_API_KEY` (matching the service's own `ADEL_API_KEY`, sent as `X-Adel-API-Key` ).

---

## Keeping the two in sync (optional, later)

---

If you keep `captadel/` here as a mirror, you can move changes across with the same prefix:

```
# push new flygaca-side captadel/ commits up to the standalone repo:
git subtree push --prefix=captadel https://github.com/iflygaca/Captain-Adel.git main

# pull standalone changes back into the monorepo subtree:
git subtree pull --prefix=captadel https://github.com/iflygaca/Captain-Adel.git main --squash
```

Pick one as the source of truth (the standalone repo is the natural choice once captadel.com is live) to avoid divergence.